Name: Yogita Dinesh Patil
D20A
57

EXPERIMENT 6

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE**.**

THEORY

1. Smart Contract

A smart contract is a program deployed on a blockchain that automatically executes predefined rules when its functions are called. In this experiment, a Ballot/Voting smart contract is created using Solidity and deployed using Remix IDE.

The Ballot contract allows:

- The chairperson to grant voting rights to voters.
- Voters to cast their votes.
- Voters to delegate their votes to other voters.
- The contract to calculate the winning proposal.
- The name of the winning proposal to be displayed.

2. Require Statement

The `require` statement is used as a guard condition in Solidity. It checks whether a particular condition is true before allowing a function to execute.

If the condition is false, the function execution stops and the transaction is reverted. Any changes made during that transaction are rolled back.

It is used to ensure that only authorized users can perform operations and to prevent invalid transactions. Thus, it improves the security and reliability of smart contracts.

3. Mapping

A mapping is a Solidity data structure that associates keys with values, similar to a hash table. In the Ballot contract, mapping connects Ethereum addresses with their corresponding voter information.

The voter information includes voting weight, voting status, delegated address and proposal index. Mappings provide efficient data lookup but do not have a length property and cannot be directly iterated over.

4. Storage

Storage refers to permanent data stored on the blockchain. Variables stored in storage remain available across transactions until they are modified.

In the Ballot contract, voter information is maintained in storage throughout the contract's lifecycle. However, storing data on the blockchain consumes gas, making storage more expensive.

5. Memory

Memory is temporary data storage used during function execution. It is mainly used for temporary variables, function arguments and computations.

The data stored in memory is discarded once the function execution is completed. Compared to storage, memory is less expensive and does not permanently store information on the blockchain.

6. Why bytes32 Instead of string?

`bytes32` is a fixed-size data type that stores exactly 32 bytes of data. It provides efficient storage, faster comparisons and lower gas consumption. However, it limits the proposal name to 32 bytes.

On the other hand, `string` is a dynamically sized data type that supports variable-length text. It is more flexible and readable but may require more gas and complex handling.

Therefore, `bytes32` is preferred when gas efficiency is important, whereas `string` is preferred for flexibility and readability.

7. Ballot Contract

The Ballot contract implements a voting mechanism using Solidity. It manages voters, voting rights, vote delegation and proposal selection. It also calculates the winning proposal based on the total votes received and displays the winner's name.

Output & Code:

Structure of Voter and Proposal

```
struct Voter {
    uint weight;
    bool voted;
    address delegate;
    uint vote;
}

struct Proposal {
    string name;
    uint voteCount;
}
```

Constructor

```
constructor(string[] memory proposalNames) {
    chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint i = 0; i < proposalNames.length; i++) {
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}
```

Give Right to Vote

```
function giveRightToVote(address voter) external {
    require(msg.sender == chairperson,
        "Only chairperson can give right to vote.");

    require(!voters[voter].voted,
        "The voter already voted.");

    require(voters[voter].weight == 0,
        "Voter already has the right to vote.");

    voters[voter].weight = 1;
}
```

Vote Delegation

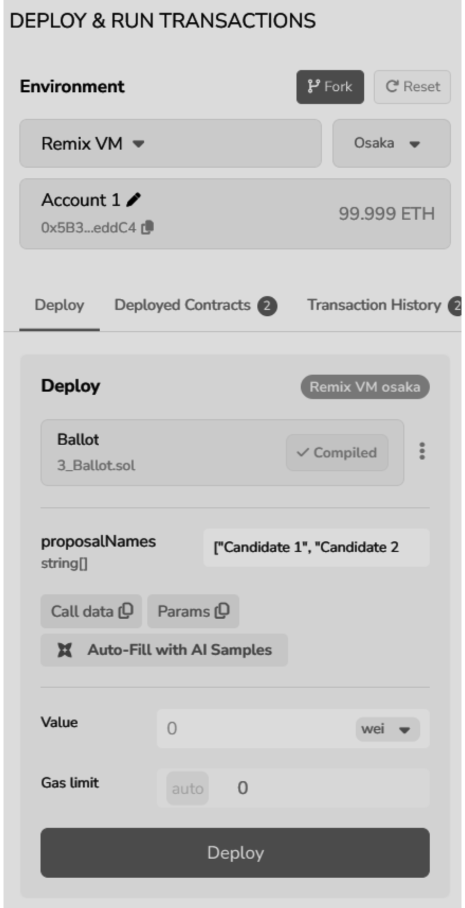
```
function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1);

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}
```



Environment: Remix VM, Osaka

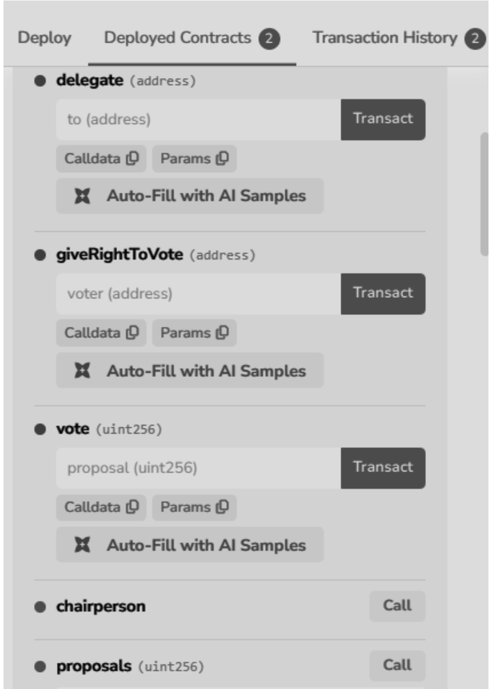
Account 1: 0x5B3...eddC4, 99.999 ETH

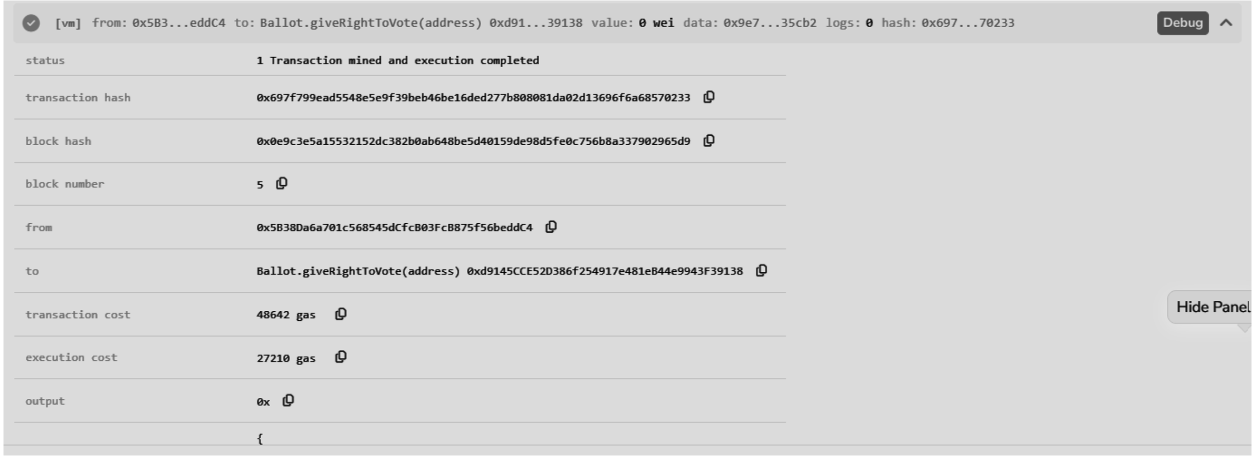
Ballot: 3_Ballot.sol

proposalNames string[]: ["Candidate 1", "Candidate 2

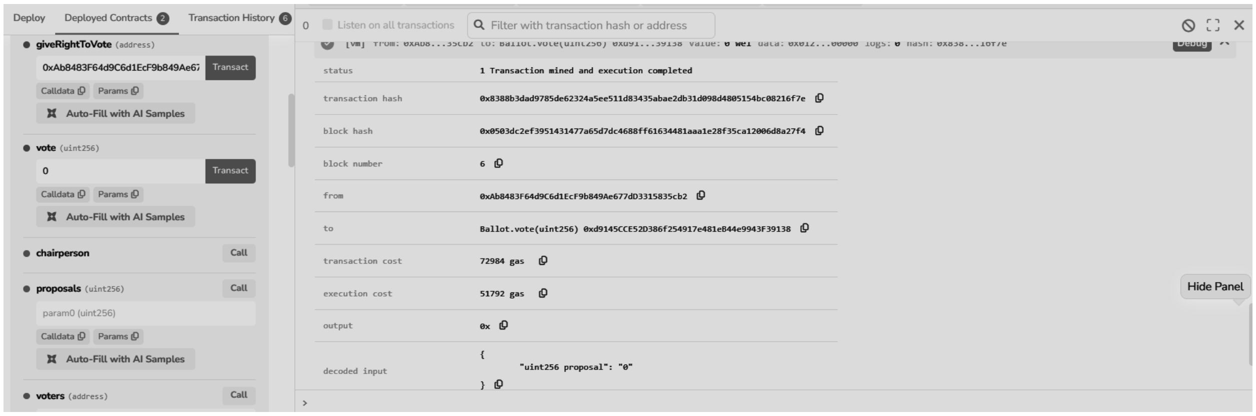
Value: 0 wei

Gas limit: auto 0





```
[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xd91...39138 value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x697...70233
status              1 Transaction mined and execution completed
transaction hash    0x697f799ead5548e5e9f39beb46be16ded277b808081da02d13696f6a68570233
block hash          0x0e9c3e5a15532152dc382b0ab648be5d40159de98d5fe0c756b8a337902965d9
block number        5
from                0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to                  Ballot.giveRightToVote(address) 0xd9145CCE52D386f254917e481eB44e9943F39138
transaction cost    48642 gas
execution cost      27210 gas
output              0x
                    {
```


Deploy
Deployed Contracts 2
Transaction History 6
Transact
Calldata
Params
Auto-Fill with AI Samples
Call
Listen on all transactions
Filter with transaction hash or address
Debug
Hide Panel

giveRightToVote (address): 0xAb8483F64d9C6d1EcF9b849Ae67

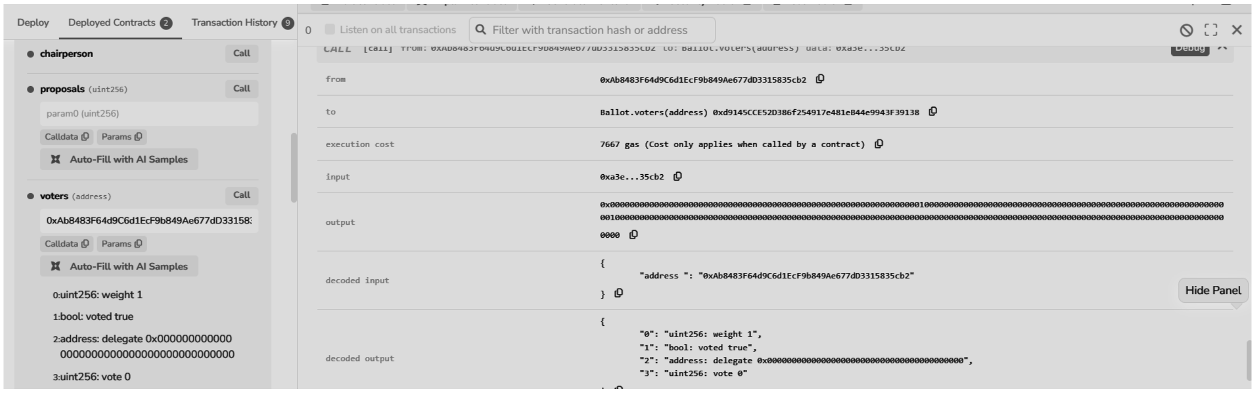
vote (uint256): 0

chairperson

proposals (uint256): param0 (uint256)

voters (address)

```
[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00000 logs: 0 hash: 0x838...16f7e
status              1 Transaction mined and execution completed
transaction hash    0x8388b3dad9785de62324a5ee511d83435abae2db31d098d4805154bc08216f7e
block hash          0x0503dc2ef3951431477a65d7dc4688ff61634481aaa1e28f35ca12006d8a27f4
block number        6
from                0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
to                  Ballot.vote(uint256) 0xd9145CCE52D386f254917e481eB44e9943F39138
transaction cost    72984 gas
execution cost      51792 gas
output              0x
decoded input       {
                            "uint256 proposal": "0"
                    }
```


Deploy
Deployed Contracts 2
Transaction History 9
Call
Calldata
Params
Auto-Fill with AI Samples
Listen on all transactions
Filter with transaction hash or address
Debug
Hide Panel

chairperson

proposals (uint256): param0 (uint256)

voters (address): 0xAb8483F64d9C6d1EcF9b849Ae677dD33158

0:uint256: weight 1

1:bool: voted true

2:address: delegate 0x0000000000000000000000000000000000000000

3:uint256: vote 0

```
CALL  [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 to: Ballot.voters(address) data: 0xa3e...35cb2
from                0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
to                  Ballot.voters(address) 0xd9145CCE52D386f254917e481eB44e9943F39138
execution cost      7667 gas (Cost only applies when called by a contract)
input               0xa3e...35cb2
output              0x0000000000000000000000000000000000000000000000000000000000000001000000000000000000000000000000000000000000000000000000000000000
                    0010000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
                    0000
decoded input       {
                            "address ": "0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2"
                    }
decoded output      {
                            "0": "uint256: weight 1",
                            "1": "bool: voted true",
                            "2": "address: delegate 0x0000000000000000000000000000000000000000",
                            "3": "uint256: vote 0"
```

Conclusion

Successfully developed and deployed an ETH Transfer smart contract using Solidity on Remix IDE. MetaMask was integrated to perform ETH transactions on the Sepolia Testnet. The transfer was successfully executed and verified using Sepolia Etherscan. Thus, the working of smart contracts and blockchain transactions was understood.